

Transfer Learning

Módulo 4 · Deep Learning

Feature extraction vs fine-tuning, differential learning rate, árbol de decisión por tamaño de dataset y código práctico en PyTorch y Hugging Face.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Transfer Learning

Entrenar GPT-3 desde cero costó aproximadamente 12 millones de dólares en cómputo de nube. Ninguna empresa mediana puede permitirse eso. El transfer learning es la solución que democratiza el acceso a las capacidades de los modelos más potentes: en lugar de entrenar desde cero, tomamos un modelo que ya aprendió representaciones ricas durante semanas en hardware masivo y lo adaptamos a nuestro caso de uso específico con horas o días de entrenamiento en hardware modesto.

El transfer learning es la técnica más transformadora del deep learning aplicado en empresa. PyTorch oficial documenta que "es relativamente raro tener un dataset de tamaño suficiente como para entrenar una red desde cero". La documentación de TensorFlow va más allá: "es una buena práctica utilizar modelos preentrenados como base". Un estudio empírico en la conferencia ICLR demostró que el transfer learning reduce los requisitos de datos en un 80-90%, permitiendo que startups con datasets pequeños alcancen rendimientos de nivel empresarial.

Las dos estrategias principales de transfer learning

Feature extraction: el modelo como extractor fijo de características

En feature extraction, el modelo preentrenado actúa como un extractor de características fijo: sus pesos se congelan (para `param` in `model.parameters()`: `param.requires_grad = False`) y solo se entrena la nueva "cabeza" del modelo (las últimas capas adaptadas al nuevo problema). Las capas congeladas transforman las entradas en representaciones ricas que la cabeza nueva aprende a clasificar.

Feature extraction es la estrategia correcta cuando el dataset de destino es pequeño (menos de 1.000 ejemplos por clase para visión) y el dominio del problema es similar al dominio del preentrenamiento. Los riesgos de sobreajuste son mucho menores porque el número de parámetros que se entrena es pequeño. El coste computacional también es mucho menor: no hay que calcular gradientes para las capas congeladas.

Fine-tuning completo: adaptar toda la red

En fine-tuning completo, todos los pesos del modelo se actualizan durante el entrenamiento en el nuevo dominio, con una tasa de aprendizaje generalmente mucho menor que en el entrenamiento original (10-100x menor para evitar "catastrophic forgetting", la pérdida de las representaciones preaprendidas). El fine-tuning completo es la estrategia correcta cuando el dataset de destino es grande (decenas de miles de ejemplos) o cuando el dominio del nuevo problema es muy diferente del preentrenamiento.

Fine-tuning por capas (layer-wise fine-tuning)

Una estrategia intermedia muy efectiva es el fine-tuning progresivo: empieza con feature extraction (solo la cabeza), después descongela gradualmente las últimas capas del backbone y

entrena con una tasa de aprendizaje muy pequeña, y solo si el rendimiento es insuficiente descongelar capas más profundas. Las capas más profundas del modelo aprenden características más genéricas (bordes, texturas básicas en visión; patrones sintácticos básicos en NLP) que son útiles para prácticamente cualquier dominio y raramente necesitan actualizarse.

SECCIÓN 3 · CÓMO FUNCIONA

Implementación en PyTorch: código y patrones

El patrón estándar en PyTorch para transfer learning de visión: cargar el modelo preentrenado (`model = torchvision.models.resnet50(weights="IMAGENET1K_V2")`), congelar todas las capas (`for param in model.parameters(): param.requires_grad = False`), reemplazar la última capa con una nueva adaptada al número de clases del nuevo problema (`model.fc = nn.Linear(model.fc.in_features, num_classes)`), definir un optimizador que solo actualice los parámetros `requires_grad=True` (`optimizer = torch.optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)`), y entrenar.

Para NLP con Hugging Face, el patrón es: `from transformers import AutoModel, AutoTokenizer`, `model = AutoModel.from_pretrained("bert-base-uncased")`, agregar una cabeza de clasificación (`model.classifier = nn.Linear(model.config.hidden_size, num_labels)`), y entrenar con la clase `Trainer` de Hugging Face o con un ciclo de entrenamiento manual en PyTorch. La tasa de aprendizaje estándar para fine-tuning de BERT y sus variantes es $2e-5$ a $5e-5$.

El diagnóstico de un fine-tuning problemático: si la pérdida de validación no baja desde el principio → tasa de aprendizaje demasiado alta o modelo mal adaptado. Si la pérdida de entrenamiento baja pero la de validación sube desde el principio → demasiados parámetros entrenables para el tamaño del dataset (añade más congelamiento). Si el rendimiento es peor que el modelo base sin adaptar → posible error en el preprocesamiento de los datos.

SECCIÓN 4 · EJEMPLOS REALES

- Clasificación de imágenes médicas con ResNet50 (hospital): con 800 imágenes etiquetadas de histología de tumor pulmonar (400 maligno, 400 benigno), fine-tuning de ResNet50 preentrenado en ImageNet produce AUC-ROC de 0.94 en 2 horas de entrenamiento en una GPU de gama media. Entrenar desde cero con 800 imágenes produciría AUC-ROC de ~0.65, insuficiente para uso clínico.
- Clasificación de emails de soporte técnico con BERT (empresa tecnológica): con 3.000 emails etiquetados en 8 categorías, fine-tuning de BERT base uncased produce accuracy de 91% en 30 minutos en una GPU. Un clasificador de ML clásico sobre TF-IDF produce 75% de accuracy con más tiempo de preparación de datos.
- Detección de objetos en línea de manufactura con EfficientDet (Bosch): con 2.000 imágenes de defectos etiquetados, fine-tuning de EfficientDet preentrenado en COCO produce mAP de 0.89 en detección de defectos específicos de la línea de producción. Sin preentrenamiento, el mismo modelo con los mismos datos produce mAP de 0.47.
- Modelo de embedding especializado en dominio legal (Harvey AI): fine-tuning de un sentence-transformer sobre pares de documentos legales similares/diferentes del dominio.

El modelo especializado mejora la recuperación en sistemas RAG legales en un 25-35% respecto a modelos genéricos.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar la misma tasa de aprendizaje para el backbone preentrenado y la nueva cabeza. El backbone necesita tasas de aprendizaje muy pequeñas ($1e-5$ a $5e-5$) para no destruir las representaciones preaprendidas. La nueva cabeza puede usar tasas mayores ($1e-3$ a $1e-4$). El differential learning rate (tasas distintas por capas) es la técnica estándar.

Error 2: No adaptar el preprocesamiento al modelo preentrenado. Los modelos preentrenados esperan inputs con características específicas: normalización con la media/std del preentrenamiento para visión, tokenización con el tokenizador del modelo para NLP. Usar un preprocesamiento diferente produce representaciones incompatibles con los pesos preentrenados.

Error 3: Fine-tuning completo con dataset pequeño. Si fine-tuneas todos los pesos de ResNet50 (25M parámetros) con 500 imágenes, el sobreajuste es inevitable. Con datasets pequeños, feature extraction o fine-tuning de solo las últimas capas son las estrategias correctas.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El árbol de decisión para elegir estrategia de transfer learning: ¿tienes <1.000 ejemplos? → feature extraction. ¿Tienes 1.000-10.000 ejemplos? → fine-tuning de las últimas 1-3 capas. ¿Tienes >10.000 ejemplos? → fine-tuning completo con tasa de aprendizaje baja. ¿El dominio es muy diferente del preentrenamiento (por ejemplo, imágenes de rayos X vs ImageNet)? → fine-tuning completo independientemente del tamaño del dataset.

Para visión, el repositorio timm (pytorch-image-models) de Hugging Face tiene más de 1.000 modelos preentrenados con sus pesos. Para NLP, el Hugging Face Hub tiene más de 500.000 modelos. En ambos casos, el proceso es el mismo: busca el modelo preentrenado más adecuado para tu dominio y tamaño de problema, cárgalo con `from_pretrained`, adapta la cabeza, y aplica la estrategia de fine-tuning correcta.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T6 (Fine-tuning de modelos LLM): este tema cubre el transfer learning general. El tema siguiente lo especializa en LLMs con PEFT, LoRA y QLoRA.
- M4-T2 (CNNs): el transfer learning es la estrategia estándar para cualquier proyecto de visión artificial con CNNs.
- M5 (LLMs): los LLMs son el producto del transfer learning a mayor escala de la historia: el preentrenamiento crea la base, el RLHF adapta el comportamiento.

SECCIÓN 8 · RESUMEN EJECUTIVO

Transfer learning toma representaciones aprendidas en un problema grande (ImageNet, corpus de texto masivo) y las adapta a un nuevo problema con pocos datos. Las dos estrategias son feature extraction (congelar todo excepto la cabeza, ideal para <1K

ejemplos) y fine-tuning (actualizar todo con tasa de aprendizaje muy baja, ideal para >10K ejemplos). La estrategia intermedia (fine-tuning de las últimas capas) funciona bien entre 1K y 10K ejemplos. Herramientas: torchvision.models, timm y Hugging Face para modelos preentrenados. Differential learning rate: tasa pequeña para el backbone, tasa mayor para la nueva cabeza. Reduce los requisitos de datos en 80-90% respecto al entrenamiento desde cero.